

11 강

머신러닝

커널 기법 1

통계·데이터과학과 박준희



✧ 학습목차

- 1 서포트 벡터 분류
- 2 SVM 최적화 전략
- 3 SVM 파이썬 실습



1

서포트 벡터 분류



서포트 벡터 분류

● 희소 커널 머신(Sparse Kernel Machines)

트레이닝 데이터

$$D = \{(x_n, t_n)\}_{n=1}^N$$

여기서 $x_n \in \mathbb{R}^d$ 는 입력 벡터, $t_n \in \{-1, +1\}$ 의 레이블(target)이다.

Sparse Kernel Machines에서는 N 개 데이터 모두를 사용하기보다는 **희소(sparse)** 형태의 해를 찾는 것이 중요하다.

● 희소 커널 머신(Sparse Kernel Machines)

커널 머신(Kernel Machine)의 결정 함수는 일반적으로 다음과 같은 형태로 표현된다.

$$y(x) = \sum_{n=1}^N c_n k(x_n, x) + b$$

여기서

- c_n 는 학습 과정에서 결정되는 계수(coefficient)로 이진분류의 경우에는 뒤에서 설명할 $\alpha_n t_n$ 값과 같음
- $k(\cdot, \cdot)$ 는 커널 함수(kernel function)
- b 는 바이어스(bias)
- $\{x_n\}_{n=1}^N$ 는 트레이닝 데이터 세트의 입력 벡터
- $x \in \mathbb{R}^d$ 는 테스트 입력 벡터

서포트 벡터 분류

● 희소 커널 머신(Sparse Kernel Machines)

결정 함수(Decision Function)

$$y(x) = \mathbf{w}^T \phi(x) + b$$

- $\phi(x)$ 는 입력 x 를 어떤(고차원) 특징 공간(feature space)으로 매핑하는 함수, 즉, 특징 함수(feature map)
- $\mathbf{w}$ 와 b 는 분류 경계를 정의하는 파라미터

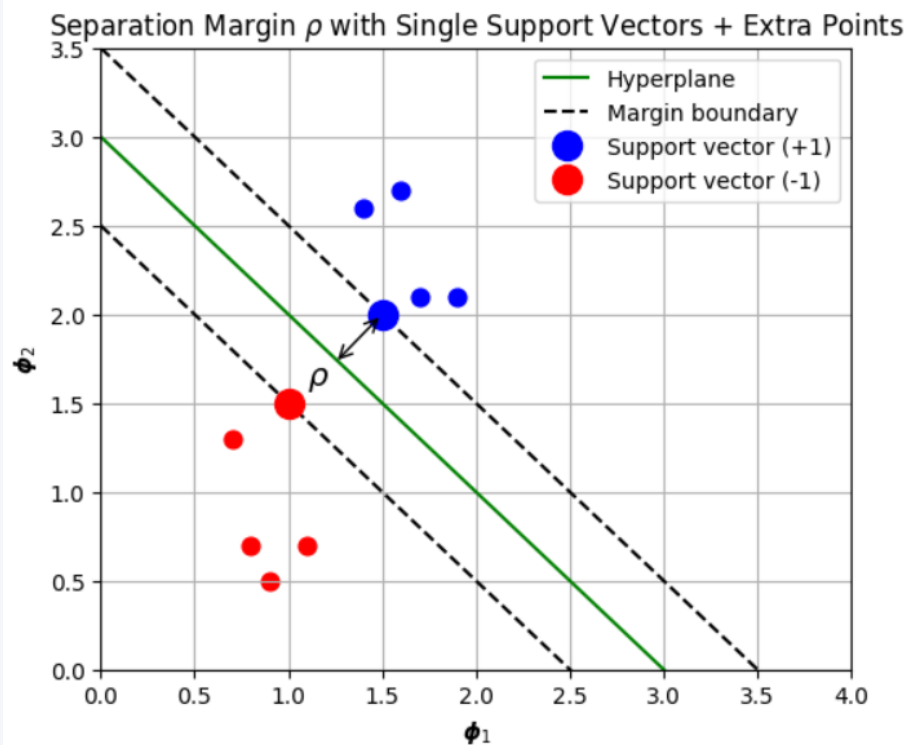
선형으로 구분 가능(linearly separable)이라 가정하면, 다음을 만족하는 $\mathbf{w}$ 와 b 가 존재한다.

$$\begin{aligned} \mathbf{w}^T \phi(x_n) + b &> 0 \quad \text{if } t_n = +1, \\ \mathbf{w}^T \phi(x_m) + b &< 0 \quad \text{if } t_m = -1. \end{aligned}$$

서포트 벡터 분류

● Separation Margin ρ

- 분류 초평면(하이퍼플레인, hyperplane)과 데이터 샘플 간의 최소 거리 ρ 를 **separation margin**이라 한다.



출처:파이썬 프로그램 실습내용 중

1

서포트 벡터 분류

● 최대 마진 분류 전략

캐노니컬 표현(canonical representation)은 서포트 벡터에서 결정함수의 출력이 +1 혹은 -1값을 갖도록 스케일링한 표현인데, 이 경우에는 모든 n 에 대해

$$t_n \times (\mathbf{w}^T \phi(x_n) + b) \geq 1$$

이 성립한다.

이때, 데이터 중 경계에 있는 항(Support Vectors)에 대해 $t_n \times (\mathbf{w}^T \phi(x_n) + b) = 1$ 이 된다.

1

서포트 벡터 분류

● 최대 마진 분류 전략

- 이 캐노니컬 표현 하에서 separation margin ρ 는

$$\rho = \frac{1}{\|\mathbf{w}\|}$$

로 주어진다 (점과 초평면 사이의 거리 공식 참고).

- 머신러닝 모델의 관점에서 이 separation margin을 늘리는 것은 다음과 같은 의미를 갖는다고 볼 수 있다:
 - 더 나은 일반화(generalization) 성능을 제공할 수 있도록 만든다.
 - 이상치(outlier)에 대해 강건성(robustness)을 지니도록 해준다.

2

SVM 최적화 전략



2 SVM 최적화 전략

● 최대 마진을 찾는 최적화문제

- Quadratic Program 최적화문제

$$\begin{aligned} \max_{\mathbf{w}, b} \quad & \rho = \frac{1}{\|\mathbf{w}\|} \\ \text{subject to} \quad & t_n \times (\mathbf{w}^T \phi(x_n) + b) \geq 1 \quad \text{for all } n = 1, \dots, N. \end{aligned}$$

2 SVM 최적화 전략

● 최대 마진을 찾는 최적화문제

- Quadratic Program 최적화문제

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2$$

subject to $1 - t_n(\mathbf{w}^T \phi(x_n) + b) \leq 0, \quad \text{for all } n = 1, \dots, N.$

- $\mathbf{w} \in \mathbb{R}^d$ 와 $b \in \mathbb{R}$ 는 결정 초평면의 파라미터
- $\phi(\cdot)$ 는 특징 공간(feature space)으로의 매핑하는 특징 함수(feature map)
- $t_n \in \{+1, -1\}$ 는 데이터 타겟(레이블)

2 SVM 최적화 전략

● 라그랑주 승수법

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{n=1}^N \alpha_n \left[1 - t_n (\mathbf{w}^T \phi(x_n) + b) \right],$$

$$\alpha = [\alpha_1, \alpha_2, \dots, \alpha_N]^T$$

여기서, α_n 은 n 번째 부등식 제약 조건 $1 - t_n (\mathbf{w}^T \phi(x_n) + b) \leq 0$ 에 대응하는 라그랑주 승수로, $\alpha_n \geq 0$ 를 만족해야 한다.

2 SVM 최적화 전략

● Karush-Kuhn-Tucker (KKT) 조건

- Stationarity 조건: 라그랑주 함수의 미분값이 0

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0, \quad \frac{\partial \mathcal{L}}{\partial b} = 0.$$

- Primal Feasibility : 프라이멀 문제의 제약조건 충족

$$1 - t_n (\mathbf{w}^T \phi(x_n) + b) \leq 0, \quad n = 1, \dots, N.$$

2 SVM 최적화 전략

● Karush-Kuhn-Tucker (KKT) 조건

- Dual Feasibility : 듀얼 문제의 제약조건 충족

$$\alpha_n \geq 0, \quad n = 1, \dots, N.$$

- Complementary Slackness : 프라이멀/듀얼 상보성

$$\alpha_n [1 - t_n(\mathbf{w}^T \phi(x_n) + b)] = 0, \quad n = 1, \dots, N.$$

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

$$\begin{aligned} \min_{x_1, x_2} \quad & f(x_1, x_2) = x_1^2 + x_2^2 \\ \text{subject to} \quad & x_1 \geq 1, \\ & x_2 \geq 2. \end{aligned}$$

- 목적함수(2차식): $f(x) = x_1^2 + x_2^2$ 는 $\mathbf{x}$ 부터 원점까지의 거리를 제곱한 값
- 제약조건(1차 부등식): $x_1 \geq 1, x_2 \geq 2$ 를 만족해야 한다.

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

```
import numpy as np
from scipy.optimize import minimize

def objective(x):
    """
    목적함수:  $f(x) = x_1^2 + x_2^2$ 
    """
    return x[0]**2 + x[1]**2
```

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

```
# 제약:  $x_1 \geq 1$ ,  $x_2 \geq 2$   
# -> scipy에서는 'ineq' 타입으로  $fun(x) \geq 0$  형태가 되어야 하므로  
#  $x_1 - 1 \geq 0$ ,  $x_2 - 2 \geq 0$  로 작성  
constraints = [  
    {'type': 'ineq', 'fun': lambda x: x[0] - 1},  
    {'type': 'ineq', 'fun': lambda x: x[1] - 2},  
]
```

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

```
# 초기값 설정
x0 = np.array([0.0, 0.0])

# 최적화 수행
solution = minimize(
    fun=objective,      # 목적함수
    x0=x0,              # 초기값
    constraints=constraints # 제약조건
)
```

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

```
# 결과 출력  
print("최적해 (x1, x2):", solution.x)  
print("최적값 f(x1, x2):", solution.fun)  
print("성공 여부:", solution.success)
```

```
최적해 (x1, x2): [1. 2.]  
최적값 f(x1, x2): 4.9999999999999997  
성공 여부: True
```

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

- Stationarity 조건: 라그랑주함수의 미분값이 0

$$\mathcal{L}(x_1, x_2, \alpha_1, \alpha_2) = x_1^2 + x_2^2 + \alpha_1(1 - x_1) + \alpha_2(2 - x_2),$$

$$\frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 - \alpha_1 = 0, \quad \frac{\partial \mathcal{L}}{\partial x_2} = 2x_2 - \alpha_2 = 0.$$

- Primal Feasibility : 프라이멀 문제의 제약조건 충족

$$g_1(x_1, x_2) = 1 - x_1 \leq 0, \quad g_2(x_1, x_2) = 2 - x_2 \leq 0.$$

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

- Dual Feasibility : 듀얼 문제의 제약조건 충족

$$\alpha_1 \geq 0, \quad \alpha_2 \geq 0.$$

- Complementary Slackness : 프라이멀/듀얼 상보성

$$\alpha_1 (1 - x_1) = 0, \quad \alpha_2 (2 - x_2) = 0.$$

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

$x_1 = 1, x_2 = 2$ 일 때,

- **Stationarity:** 위에서 구한 값으로 $\nabla_{\mathbf{x}} \mathcal{L} = 0$ 성립
- **Primal feasibility:** $x_1 \geq 1, x_2 \geq 2$ 성립
- **Dual feasibility:** $\alpha_1 = 2 \geq 0, \alpha_2 = 4 \geq 0$ 성립
- **Complementary slackness:** $\alpha_1(1 - 1) = 2 \cdot 0 = 0,$
 $\alpha_2(2 - 2) = 4 \cdot 0 = 0$ 성립

2 SVM 최적화 전략

● Quadratic Program & KKT 조건 - 직관적 예제

결과적으로, KKT 조건을 만족하는 해는

$$x_1^* = 1, \quad x_2^* = 2, \quad \alpha_1^* = 2, \quad \alpha_2^* = 4.$$

여기서, 두 제약 모두 활성화(= 경계 위)이고, 최적값은

$$f(x_1^*, x_2^*) = (1)^2 + (2)^2 = 1 + 4 = 5.$$

2 SVM 최적화 전략

● 프라이멀 문제와 듀얼 문제

다시 원래의 (Hard Margin) SVM 문제로 돌아와보자.

$$\min_{\mathbf{w}, b} \quad \frac{1}{2} \|\mathbf{w}\|^2$$

$$\text{subject to} \quad 1 - t_n(\mathbf{w}^T \phi(x_n) + b) \leq 0, \quad \text{for all } n = 1, \dots, N.$$

위 식을 최소화하는(즉, 마진 ρ 를 최대화하는) $\mathbf{w}, b$,를 구하는 것이 **프라이멀 문제**(primal problem)이고, 이 때 $\mathbf{w}, b$,는 **프라이멀 변수**이다.

2 SVM 최적화 전략

● 프라이머 문제와 듀얼 문제

$$\mathcal{L}(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{n=1}^N \alpha_n \left[1 - t_n (\mathbf{w}^T \phi(x_n) + b) \right].$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = 0 \implies \mathbf{w} = \sum_{n=1}^N \alpha_n t_n \phi(x_n).$$

$$\frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{n=1}^N \alpha_n t_n = 0.$$

2 SVM 최적화 전략

● 프라이멀 문제와 듀얼 문제

$$\max_{\alpha} W(\alpha) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m t_n t_m \phi(x_n)^T \phi(x_m),$$

$$\text{subject to } \sum_{n=1}^N \alpha_n t_n = 0, \quad \alpha_n \geq 0 \ (n = 1, \dots, N).$$

- $\alpha = [\alpha_1, \alpha_2, \dots, \alpha_N]^T$ 는 듀얼 변수
- $W(\alpha) = \mathcal{L}(\mathbf{w}^*, b^*, \alpha)$
- $\phi(x_n)^T \phi(x_m)$ 는 커널 함수 $k(x_n, x_m)$ 로 대체 가능.

2 SVM 최적화 전략

● 커널 트릭 (Kernel Trick)

$k(x_n, x_m) = \langle \phi(x_n), \phi(x_m) \rangle$ 을 통해 고차원 feature map $\phi(x)$ 를 직접 계산하지 않고도, 위의 최적화 문제에 등장하는 $\phi(x_n)^T \phi(x_m)$ 항을 $k(x_n, x_m)$ 로 치환하여 계산 가능

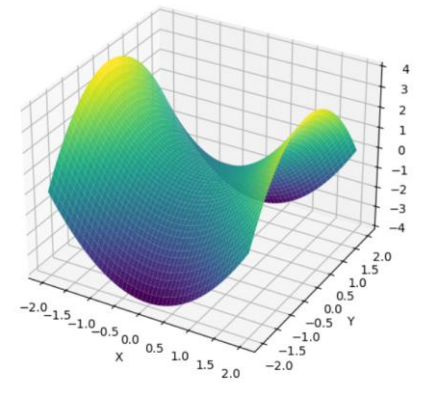
2 SVM 최적화 전략

● 커널 트릭 (Kernel Trick)

정리하면, SVM 맥락하에서 선형 제약을 가진 QP 문제(프라이멀 문제)는 다음의 Strong Duality를 만족한다 (자세한 수학적 증명은 생략).

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 = \max_{\alpha} W(\alpha)$$

그리하여, 듀얼 문제를 풀면 (즉, $W(\alpha)$ 의 최대값을 제약조건 하에서 구하면), 그 해는 프라이멀 문제의 해와 일치한다. 즉, $\frac{1}{2} \|\mathbf{w}\|^2$ 의 최소값과 일치한다. (마치, 말 안장을 측면에서 바라볼 때에는 최저점을 구하는 문제가, 후면에서 바라볼 때에는 최고점을 구하는 문제와 같고, 각 상황에서의 최적값이 서로 같다는 것과 비슷함으로 비유할 수 있다)



2 SVM 최적화 전략

● 커널 트릭 (Kernel Trick)

- Mercer's Theorem (개략적인 진술)

어떤 대칭 함수 $k(x, y)$ 가 (특정 공간에서) 정의되고,
모든 $f \in L_2$ 에 대해 $\iint k(x, y)f(x)f(y)dxdy \geq 0$ 가 성립하면,
적절한(잠재적으로 고차원 혹은 무한 차원인 입력 공간 위에 정의된)
매핑 ϕ 가 존재하여 $k(x, y) = \langle \phi(x), \phi(y) \rangle$ 가 성립한다.

2 SVM 최적화 전략

● 커널 트릭 적용 가능한 커널 함수들

RBF Kernel

- 가장 널리 사용되는 커널 중 하나:

$$K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{1}{2\sigma^2} \|\mathbf{x} - \mathbf{y}\|^2\right).$$

- σ (또는 $\gamma = 1/(2\sigma^2)$)는 커널 폭을 제어하는 하이퍼파라미터
- 무한 차원 특징 공간으로 매핑하는 효과가 있어, 매우 유연한 분류·회귀 모델링 가능.

2 SVM 최적화 전략

● 커널 트릭 적용 가능한 커널 함수들

Polynomial Kernel

$$K(\mathbf{x}, \mathbf{y}) = (\langle \mathbf{x}, \mathbf{y} \rangle + c)^d.$$

- $\langle \mathbf{x}, \mathbf{y} \rangle$ 은 $\mathbf{x}^T \mathbf{y}$ 로, 보통 입력 $\mathbf{x}, \mathbf{y}$ 의 내적
- c 는 상수항 (예: 바이어스 항을 간접적으로 표현)
- d 는 다항식의 차수(차원이 높을수록 복잡한 결정경계 가능)
- $\phi(\mathbf{x})$ 는 d 가 커짐에 따라 매우 많은 항을 가질 수 있지만, 커널 트릭 덕분에 직접 전개하지 않아도 계산 가능.

2 SVM 최적화 전략

● 커널 트릭 적용 가능한 커널 함수들

- Sigmoid Kernel:

$$K(\mathbf{x}, \mathbf{y}) = \tanh(\alpha \mathbf{x}^T \mathbf{y} + r)$$

- 신경망의 활성화함수(하이퍼볼릭 탄젠트)와 유사
- 다만 모든 상황에서 Mercer 조건을 만족하는 것은 아니므로, 파라미터에 따라 주의가 필요

- Laplacian Kernel 등등

3

SVM 파이썬 실습



3

SVM 파이썬 실습

● Hard Margin SVM - 데이터 준비

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

X = np.array([
    [1.5, 2.0],
    [1.4, 2.6],
    [1.7, 2.1],
    [1.6, 2.7],
    [1.9, 2.1],
    [1.0, 1.5],
    [0.7, 1.3],
    [0.8, 0.7],
    [0.9, 0.5],
    [1.1, 1.7]
])

t = np.array([1,1,1,1,1,-1,-1,-1,-1,-1]) # +1 / -1 classes
```

출처:파이썬 프로그램 실습내용 중

3 SVM 파이썬 실습

● Hard Margin SVM - RBF 커널 및 커널 행렬

```
def rbf_kernel(x1, x2, gamma=1.0):  
    return np.exp(-gamma * np.sum((x1 - x2)**2))  
  
def compute_kernel_matrix(X, gamma=1.0):  
    N = len(X)  
    K = np.zeros((N, N))  
    for i in range(N):  
        for j in range(N):  
            K[i, j] = rbf_kernel(X[i], X[j], gamma)  
    return K
```

3 SVM 파이썬 실습

● Hard Margin SVM - 결정 함수

```
def decision_function(alpha, b, X_train, t_train, gamma, X_test):  
    """  
     $f(x) = \sum_n \alpha_n * t_n * K(x_n, x) + b$   
    """  
    out = []  
    for x in X_test:  
        val = b  
        for i in range(len(X_train)):  
            if alpha[i] > 1e-8:  
                val += alpha[i] * t_train[i] * rbf_kernel(X_train[i], x, gamma)  
        out.append(val)  
    return np.array(out)
```

3 SVM 파이썬 실습

● Hard Margin SVM - 예측 함수, 결정 경계 시각화 함수

```
def predict(alpha, b, X_train, t_train, gamma, X_test):
    vals = decision_function(alpha, b, X_train, t_train, gamma, X_test)
    return np.sign(vals)

def plot_decision_boundary(alpha, b, X, t, gamma, title="RBF SVM"):
    x_min, x_max = X[:,0].min()-1, X[:,0].max()+1
    y_min, y_max = X[:,1].min()-1, X[:,1].max()+1
    xx, yy = np.meshgrid(
        np.linspace(x_min, x_max, 200),
        np.linspace(y_min, y_max, 200)
    )
    grid_points = np.c_[xx.ravel(), yy.ravel()]
    Z = predict(alpha, b, X, t, gamma, grid_points)
    Z = Z.reshape(xx.shape)

    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.bwr)
```

출처:파이썬 프로그램 실습내용 중

3 SVM 파이썬 실습

● Hard Margin SVM - 듀얼 목적 함수

```
def dual_objective(alpha, t, K):  
    """  
    Dual objective:  
        maximize  $W(\alpha) = \sum(\alpha_n) - 1/2 \sum_{n,m} \alpha_n \alpha_m t_n t_m K(n,m)$   
    We'll minimize the negative of  $W(\alpha)$ .  
    """  
    term1 = 0.5 * np.sum((alpha * t)[: , None] * (alpha * t)[None, :] * K)  
    term2 = np.sum(alpha)  
    return term1 - term2
```

3 SVM 파이썬 실습

● Hard Margin SVM - RBF SVM 함수

```
def fit_svm_rbf(X, t, gamma=1.0):  
    """  
    Solve the Hard-Margin SVM dual problem ( $\alpha \geq 0$ ,  $\sum(\alpha_n t_n) = 0$ ).  
    """  
    N = len(X)  
    K = compute_kernel_matrix(X, gamma=gamma)  
  
    alpha0 = np.zeros(N)  
  
    def eq_constraint(alpha):  
        return np.dot(alpha, t)  
  
    constraints = [{'type': 'eq', 'fun': eq_constraint}]  
    bounds = [(0, None)] * N
```

출처:파이썬 프로그램 실습내용 중

3 SVM 파이썬 실습

● Hard Margin SVM - RBF SVM 함수

```
res = minimize(  
    fun=dual_objective,  
    x0=alpha0,  
    args=(t, K),  
    bounds=bounds,  
    constraints=constraints,  
    options={'maxiter': 1000, 'disp': True}  
)  
alpha_opt = res.x
```

3 SVM 파이썬 실습

● Hard Margin SVM - RBF SVM 함수

```
# KKT로부터 b 계산
sv_indices = np.where(alpha_opt > 1e-8)[0]
if len(sv_indices) == 0:
    b_opt = 0.0
else:
    i = sv_indices[0] # pick one support vector
    sum_j = np.sum(alpha_opt * t * K[i, :])
    b_opt = t[i] - sum_j

return alpha_opt, b_opt
```

3

SVM 파이썬 실습

● Hard Margin SVM - 예시 실행

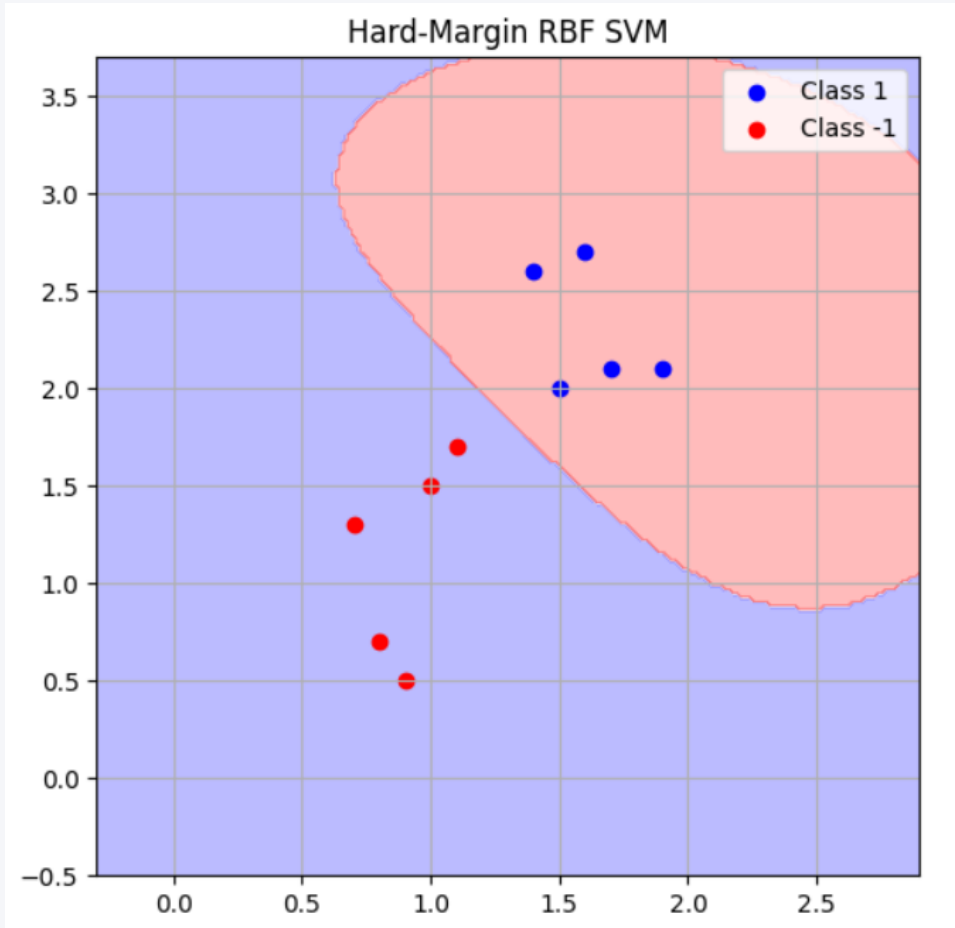
```
gamma = 1.0

# Hard-Margin SVM 결과
alpha, b = fit_svm_rbf(X, t, gamma=gamma)
print("\n alpha:", alpha)
print(" b:", b)

plt.figure(figsize=(6,6))
plot_decision_boundary(alpha, b, X, t, gamma,
                        title="Hard-Margin RBF SVM")
plt.show()
```

3 SVM 파이썬 실습

● Hard Margin SVM - 예시 실행



출처: 파이썬 프로그램 실습내용 중

★ 정리하기

■ 서포트 벡터 분류

SVM : 주어진 데이터를 가지고 결정함수의 마진을 최대화

■ SVM 최적화 전략

프라이멀 문제 : 원래 주어진 최적화 문제

듀얼 문제 : 라그랑지안을 통해 유도되는 새로운 문제

커널 트릭 : 계산이 어려운 특징 함수 간 내적 연산 대체

KKT 조건 : 제약이 있는 최적화 문제의 해를 구하는 방법

■ SVM 파이썬 실습

RBF 커널의 γ 값을 조절하여 SVM 성능 튜닝 가능

12 강

다음시간안내

커널 기법 2

